VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

MÃ QUỐC CƯỜNG – 522I0001
NGUYỄN CAO PHI – 521V0006

A Comparative Analysis of FPMax and GenMax for Mining Frequent Maximal Itemsets from Uncertain Transaction Databases

Information Technology Project – 504091

HO CHI MINH CITY, YEAR 2026

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

MÃ QUỐC CƯỜNG – 522I0001
NGUYỄN CAO PHI – 521V0006

A Comparative Analysis of FPMax and GenMax for Mining Frequent Maximal Itemsets from Uncertain Transaction Databases

Information Technology Project – 504091

Advised by
Prof. Nguyễn Chí Thiện

HO CHI MINH CITY, YEAR 2026

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Professor Nguyễn Chí Thiện for his patient guidance, thoughtful supervision, and continuous encouragement throughout this project. His expertise in data mining and algorithm design provided invaluable direction at every stage of our work.

We also thank our classmates for their helpful discussions and feedback, and Ton Duc Thang University for providing the academic environment and resources that made this project possible.

Ho Chi Minh City, day month year 2026

Authors

(Signature and full name)

DECLARATION OF AUTHORSHIP

I hereby declare that this thesis was carried out by my team under the guidance and supervision of Prof Nguyễn Chí Thiện; and that the work and the results contained in it are original and have not been submitted anywhere for any previous purposes. The data and figures presented in this thesis are for analysis, comments, and evaluations from various resources by my own work and have been duly acknowledged in the reference part.

In addition, other comments, reviews and data used by other authors, and organizations have been acknowledged, and explicitly cited.

I will take full responsibility for any fraud detected in my thesis. Ton Duc Thang University is unrelated to any copyright infringement caused on my work (if any).

Ho Chi Minh City, day month year
Author

[Signature and full name]

ABSTRACT

Contents

ACKNOWLEDGMENT **1**

DECLARATION OF AUTHORSHIP **2**

TABLE OF CONTENTS **5**

IMAGE TABLE **5**

List of Abbreviations **6**

1 INTRODUCTION **7**
- 1.1 What is Frequent Itemset Mining? 7
- 1.2 The Challenge of Uncertainty 7
- 1.3 Why Maximal Itemsets? 7
- 1.4 Project Objectives 8
- 1.5 Report Structure 8

2 LITERATURE REVIEW **9**

3 PROBLEM STATEMENT AND RELATED DEFINITIONS **10**
- 3.1 Uncertain Transaction Database 10
- 3.2 Expected Support 10
- 3.3 Anti-Monotonicity Property 11
- 3.4 Frequent and Maximal Frequent Itemsets 11
- 3.5 Problem Statement 11

4 METHODS **12**
- 4.1 Branch-and-Bound Framework 12
  - 4.1.1 Pruning 1: Anti-Monotonicity Bound 12
  - 4.1.2 Pruning 2: Progressive Focusing 12
  - 4.1.3 Pruning 3: Top-K Dynamic Threshold Raising 12
- 4.2 Algorithm 1: UGenMax 13
  - 4.2.1 Data Structure: Weighted Tidsets 13
  - 4.2.2 Enumeration Strategy 13

4.2.3 Pseudocode 14
4.2.4 Complexity Analysis 14
4.3 Algorithm 2: UFPMax 15
4.3.1 Data Structure: Projected Transactions 15
4.3.2 Conditional Database Construction 15
4.3.3 Pseudocode 15
4.3.4 Complexity Analysis 16
4.4 Comparison of UGenMax and UFPMax 16
4.5 Top-K Seeding 17
4.6 Maximality Checking 17

5 EXPERIMENT DESIGN 18
5.1 Implementation Details 18
5.2 Datasets 18
5.3 Configurations 18
5.4 Correctness Validation 18
5.5 Performance Measurement 19

List of Figures

LIST OF ABBREVIATIONS

Abbreviation	Definition
FIM	Frequent Itemset Mining
MFI	Maximal Frequent Itemset
DFS	Depth-First Search
expSup	Expected Support
minsup	Minimum Expected Support Threshold
WT	Weighted Tidset
DB	Database

CHAPTER 1
INTRODUCTION

1.1 What is Frequent Itemset Mining?

Imagine a supermarket analyzing thousands of customer receipts to discover which products are frequently bought together. If many customers buy bread and butter in the same trip, the pair {bread, butter} is a **frequent itemset**. This information is valuable for product placement, cross-selling, and inventory management.

More formally, given a database of transactions (where each transaction is a set of items) and a minimum support threshold, the goal of frequent itemset mining is to find all item groups that appear in at least that many transactions. This problem was first formalized by Agrawal, Imielinski, and Swami [1] and has since become one of the most studied problems in data mining, with applications in bioinformatics, text mining, web usage analysis, network security, and recommendation systems.

1.2 The Challenge of Uncertainty

Classical algorithms assume perfect data: an item is either in a transaction or it is not. In practice, data is often imperfect. Consider these real-world scenarios:

- **Sensor networks.** A temperature sensor reports values with a known error margin. When the system logs that a sensor detected item A, it might only be 85% confident in that reading.
- **Data integration.** When merging customer records from two different databases, some matches are ambiguous. A merged record might indicate that a customer purchased item B, but with only 70% confidence due to name/address mismatches.
- **Privacy preservation.** To protect user privacy, a system deliberately adds random noise to transaction data. Each item's presence is now probabilistic rather than certain.

In all these cases, the result is an **uncertain transaction database** where each item in each transaction carries a probability between 0 and 1. The challenge is: how do we mine meaningful patterns from data that we are not entirely sure about?

1.3 Why Maximal Itemsets?

A database with m distinct items has 2^m possible subsets. Even a modest database with 100 items has more possible subsets than atoms in the universe. While most of these are not frequent, the number of frequent itemsets can still be enormous—potentially millions or more.

Maximal frequent itemsets solve this information overload problem. A frequent itemset is maximal if no proper superset of it is also frequent. The key insight is that every frequent itemset is a subset of at least one maximal frequent itemset, so the maximal set is a **lossless compression** of the full set of frequent itemsets.

1.4 Project Objectives

This project has four objectives:

1. Formally define the problem of mining frequent maximal itemsets from uncertain databases using the expected support model, with precise mathematical definitions for each concept.
2. Design and implement two algorithms (UGenMax and UFPMax) that solve this problem using branch-and-bound strategies.
3. Implement both algorithms as a pure Java SE command-line application with no external library dependencies.
4. Experimentally evaluate and compare both algorithms on benchmark datasets of varying sizes and uncertainty levels.

1.5 Report Structure

The remainder of this report is organized as follows. Chapter 2 reviews related work. Chapter 3 provides formal problem definitions. Chapter 4 presents the two algorithms in detail, including pseudocode, worked examples, and complexity analysis. Chapter 5 describes the experimental setup. Chapter 6 presents results. Chapter 7 concludes with a summary and future work.

CHAPTER 2
LITERATURE REVIEW

CHAPTER 3
PROBLEM STATEMENT AND RELATED DEFINITIONS

This section provides precise mathematical definitions for each concept. We build the definitions incrementally, starting from the data model and working up to the problem statement.

3.1 Uncertain Transaction Database

Definition 3.1 (Item Set)**.** *Let $I = \{i_1, i_2, \ldots, i_m\}$ be a finite set of m distinct items.*

Definition 3.2 (Uncertain Transaction)**.** *An uncertain transaction t_k is a set of items from I, where each item i is associated with an* ***existential probability*** *$P(i, t_k)$ satisfying:*

$$0 < P(i, t_k) \leq 1$$

Items are assumed to be mutually independent within each transaction.

Definition 3.3 (Uncertain Transaction Database)**.** *An uncertain transaction database $D = \{t_1, t_2, \ldots, t_n\}$ is an ordered collection of n uncertain transactions.*

Running Example. Throughout this report, we use the following 5-transaction database for illustration:

Table 3.1: The contextUncertain dataset

Transaction	Items (with existential probabilities)			
t_1	1(0.8)	3(0.6)	4(0.9)	
t_2	2(0.7)	3(0.5)	5(0.9)	
t_3	1(0.6)	2(0.8)	3(0.7)	5(0.85)
t_4	2(0.5)	5(0.6)		
t_5	1(0.9)	2(0.75)	3(0.8)	5(0.7)

3.2 Expected Support

Definition 3.4 (Expected Support)**.** *The expected support of an itemset $X = \{x_1, x_2, \ldots, x_p\}$ in database D is:*

$$\textit{expSup}(X) = \sum_{t \in D} \prod_{x \in X} P(x, t) \tag{3.1}$$

Worked Example. Consider $X = \{1, 3\}$:

$$
\begin{aligned}
&t_1 : P(1, t_1) \times P(3, t_1) = 0.8 \times 0.6 = 0.48 \\
&t_2 : \text{item 1 absent} = 0.00 \\
&t_3 : P(1, t_3) \times P(3, t_3) = 0.6 \times 0.7 = 0.42 \\
&t_4 : \text{item 1 absent} = 0.00 \\
&t_5 : P(1, t_5) \times P(3, t_5) = 0.9 \times 0.8 = 0.72
\end{aligned}
$$

$$\text{expSup}(\{1, 3\}) = 0.48 + 0.00 + 0.42 + 0.00 + 0.72 = \mathbf{1.62}$$

3.3 Anti-Monotonicity Property

Theorem 3.1 (Anti-Monotonicity of Expected Support)**.** *For any itemset X and any item $y \notin X$:*

$$\textit{expSup}(X \cup \{y\}) \leq \textit{expSup}(X)$$

Chứng minh. Since $0 < P(y, t) \leq 1$ for all transactions t, each term $\prod_{x \in X} P(x, t) \cdot P(y, t) \leq \prod_{x \in X} P(x, t)$. Therefore the sum cannot increase. □

3.4 Frequent and Maximal Frequent Itemsets

Definition 3.5 (Frequent Itemset)**.** *Given threshold minsup, itemset X is **frequent** if expSup$(X) \geq$ minsup.*

Definition 3.6 (Maximal Frequent Itemset)**.** *A frequent itemset X is **maximal** if for every item $y \notin X$, expSup$(X \cup \{y\}) <$ minsup.*

3.5 Problem Statement

Input: An uncertain transaction database D and either (a) a minimum expected support threshold minsup > 0, or (b) a positive integer K.

Output: The set MFI(D, minsup) of all maximal frequent itemsets with their expected support values. In Top-K mode, the K MFIs with highest expected support.

CHAPTER 4
METHODS

This chapter presents both algorithms in detail: data structures, pseudocode, worked examples, and complexity analysis.

4.1 Branch-and-Bound Framework

Both algorithms explore the **itemset lattice** using depth-first search (DFS). The search is a tree: the root is the empty set, each node is an itemset (prefix), and each edge adds one item. Without pruning, this requires exploring 2^m subsets. Branch-and-bound makes the search tractable by computing **bounds** at each node and **pruning** subtrees that cannot yield useful results.

4.1.1 Pruning 1: Anti-Monotonicity Bound

Rule: If $\text{expSup}(\text{prefix} \cup \{\text{item}\}) < \text{minsup}$, skip this item and the entire subtree below it.

Rationale: By Theorem 3.1, every superset will have even lower expected support. This rule is responsible for the majority of pruning in practice. For UGenMax, a cheaper upper bound $\min(\text{expSup}(A), \text{expSup}(B))$ is checked before the full intersection.

4.1.2 Pruning 2: Progressive Focusing

Rule: Let P be the current prefix and C the remaining candidates. If $P \cup C \subseteq M$ for some known MFI M, prune the entire subtree.

Rationale: Any discoverable itemset would be a subset of M and therefore not maximal. This rule becomes increasingly effective as more MFIs are discovered during the search.

4.1.3 Pruning 3: Top-K Dynamic Threshold Raising

In Top-K mode, a **min-heap** of size K tracks the best results. When a new MFI is discovered, if the heap is full and the new MFI's expected support exceeds the heap minimum, the minimum is replaced. The new heap minimum becomes the updated minsup.

Feedback loop: Better results $\rightarrow$ higher threshold $\rightarrow$ more aggressive pruning $\rightarrow$ faster convergence.

Design difference: UGenMax raises the threshold dynamically during search. UFPMax does **not**, because its processing order discovers single-item MFIs early, which would prematurely block multi-item MFIs.

4.2 Algorithm 1: UGenMax

UGenMax adapts GenMax [7] for uncertain data using **bottom-up, depth-first backtracking** with **weighted tidsets**.

4.2.1 Data Structure: Weighted Tidsets

Definition 4.1 (Weighted Tidset)**.** *The weighted tidset of item i is:*

$$WT(i) = \{(t, P(i,t)) \mid t \in D,\ P(i,t) > 0\}$$

sorted by transaction ID. Expected support: expSup$(i) = \sum_{(t,p)\in WT(i)} p$.

To compute expSup$(\{A, B\})$, we **intersect** WT(A) and WT(B) using a merge-join in $O(|\text{WT}(A)| + |\text{WT}(B)|)$ time. For each common transaction t, the result is $(t, P(A,t) \times P(B,t))$.

Worked Example. Using items 1 and 3 from Table 3.1:

$$\text{WT}(1) = [(t_1, 0.8),\ (t_3, 0.6),\ (t_5, 0.9)] \quad \text{expSup} = 2.3$$
$$\text{WT}(3) = [(t_1, 0.6),\ (t_2, 0.5),\ (t_3, 0.7),\ (t_5, 0.8)] \quad \text{expSup} = 2.6$$

Intersection:

$$\begin{aligned}
&t_1 : \text{both present} \rightarrow 0.8 \times 0.6 = 0.48 \\
&t_2 : \text{only in WT}(3) \rightarrow \text{skip} \\
&t_3 : \text{both present} \rightarrow 0.6 \times 0.7 = 0.42 \\
&t_5 : \text{both present} \rightarrow 0.9 \times 0.8 = 0.72
\end{aligned}$$

$$\text{WT}(\{1,3\}) = [(t_1, 0.48),\ (t_3, 0.42),\ (t_5, 0.72)] \quad \text{expSup} = 1.62$$

4.2.2 Enumeration Strategy

Items are sorted by expected support **ascending**. Candidates are processed **most frequent first** (right to left). This ordering discovers large MFIs early, strengthening progressive focusing.

4.2.3 Pseudocode

Algorithm 1 UGenMax

Require: Uncertain database D, threshold minsup
Ensure: Set MFI of all maximal frequent itemsets

```
MFI ← ∅
for each item i in D do
    Build WT(i) by scanning all transactions
end for
Remove items where expSup(i) < minsup
Sort items by expSup ascending: [i_1, i_2, ..., i_k]
GenMax(∅, [i_1, ..., i_k], null)
return MFI
```

Algorithm 2 GenMax Procedure

```
procedure GenMax(prefix, candidates, prefixTidset)
    if prefix ∪ candidates ⊆ some MFI in MFI then                ▷ Pruning 2
        return
    end if
    hasFreqExt ← false
    for idx ← |candidates| − 1 downto 0 do
        item ← candidates[idx]
        newTidset ← Intersect(prefixTidset, WT(item))
        expSup ← Sum(newTidset.probabilities)
        if expSup < minsup then                                  ▷ Pruning 1
            continue
        end if
        hasFreqExt ← true
        newPrefix ← prefix ∪ {item}
        newCands ← {c ∈ candidates[0..idx−1] | Intersect(newTidset, WT(c)).expSup ≥
    minsup}
        if newCands = ∅ then
            AddMaximal(newPrefix, expSup)
        else
            GenMax(newPrefix, newCands, newTidset)
        end if
    end for
    if not hasFreqExt and prefix ≠ ∅ then
        AddMaximal(prefix, prefixTidset.expSup)
    end if
end procedure
```

4.2.4 Complexity Analysis

Time: Worst case $O(2^m)$ nodes, each with $O(n)$ tidset intersection. Pruning dramatically reduces explored nodes in practice.

Space: $O(m \times n)$ for weighted tidsets, $O(|\text{MFI}|)$ for results, $O(m)$ for the recursion stack.

4.3 Algorithm 2: UFPMax

UFPMax adapts FPMax [8] for uncertain data using **top-down, recursive conditional database projection**.

4.3.1 Data Structure: Projected Transactions

Each projected transaction is a triple (weight, items[], probs[]):

- **weight**: accumulated product of probabilities of all prefix items. Initially 1.0.
- **items[] and probs[]**: remaining items and probabilities, sorted by frequency-descending order.

Expected support of item i in a conditional database:

$$\text{expSup}(i \mid \text{prefix}) = \sum_{t:i \in t} \text{weight}_t \times P(i,t) \tag{4.1}$$

4.3.2 Conditional Database Construction

When extending prefix by item i:

1. Select transactions containing item i.
2. Compute new_weight = old_weight $\times P(i,t)$.
3. Project to items **before** i in header order (more frequent items).
4. Remove items with conditional expSup $<$ minsup.

4.3.3 Pseudocode

Algorithm 3 UFPMax

Require: Uncertain database D, threshold minsup
Ensure: Set MFI of all maximal frequent itemsets

```
MFI ← ∅
Compute expSup for each single item
Remove items with expSup < minsup
Sort items by expSup descending: header = [h_1, ..., h_k]
Build initial projected DB (weight = 1.0, frequent items only)
FPMAX(∅, 0.0, projDB, header)
return MFI
```

Algorithm 4 FPMax Procedure

```
procedure FPMAX(prefix, prefixExpSup, condDB, headerItems)
    if prefix ∪ headerItems ⊆ some MFI then                    ▷ Pruning 2
        return
    end if
    hasFreqExt ← false
    for idx ← |headerItems| − 1 downto 0 do
        item ← headerItems[idx]
        expSup ← Σ_{t∈condDB} weight_t × P(item, t)
        if expSup < minsup then                                ▷ Pruning 1
            continue
        end if
        hasFreqExt ← true; newPrefix ← prefix ∪ {item}
        newCondDB ← BUILDCONDITIONAL(condDB, item, idx)
        newHeader ← {h ∈ header[0..idx−1] | condExpSup(h) ≥ minsup}
        if newHeader = ∅ then
            ADDMAXIMAL(newPrefix, expSup)
        else
            FPMAX(newPrefix, expSup, newCondDB, newHeader)
        end if
    end for
    if not hasFreqExt and prefix ≠ ∅ then
        ADDMAXIMAL(prefix, prefixExpSup)
    end if
end procedure
```

4.3.4 Complexity Analysis

Time: Worst case $O(2^m)$ nodes. Conditional DB construction is $O(|\text{condDB}| \times |\text{items}|)$ per node, shrinking at deeper levels.

Space: $O(n \times m)$ for the initial projected DB. Each recursive level maintains its own conditional DB. Stack depth at most m.

4.4 Comparison of UGenMax and UFPMax

Table 4.1: Comparison of UGenMax and UFPMax

Aspect	**UGenMax**	**UFPMax**
Base algorithm	GenMax (Gouda & Zaki, 2005)	FPMax (Grahne & Zhu, 2003)
Approach	Bottom-up (vertical)	Top-down (horizontal)
Data structure	Weighted tidsets	Projected conditional DBs
Item ordering	Ascending expSup	Descending expSup
Top-K threshold	Raised dynamically	Fixed, select after search
Best for	Sparse data	Dense data

4.5 Top-K Seeding

Both algorithms use a seeding phase in Top-K mode: (1) take the top-50 most frequent items, (2) generate all single and 2-item combinations, (3) fill a Top-K heap, (4) set initial minsup to 50% of the heap minimum, (5) reset the heap and begin the main search. If fewer than K results are found, retry with 25% of current threshold, up to 3 retries.

4.6 Maximality Checking

When a leaf node is reached, the candidate MFI is checked: (1) it must not be a subset of any existing MFI, and (2) existing MFIs that are subsets of the new candidate are removed. Both checks use sorted arrays for $O(|A| + |B|)$ merge-based comparison.

CHAPTER 5
EXPERIMENT DESIGN

5.1 Implementation Details

Both algorithms are implemented in Java SE 25 with no external dependencies. The code is organized into packages: `algorithm/` (UGenMax, UFPMax, WeightedTidset), `model/` (UncertainDatabase, UncertainTransaction, Itemset, TopKHeap), `util/` (PerformanceTracker). Every class includes Javadoc documentation.

5.2 Datasets

Table 5.1: Experimental datasets

Dataset	**Trans.**	**Items**	**Prob. Distribution**	**Purpose**
contextUncertain	5	5	Manual	Hand verification
mushroom_high	8,416	120	$N(\mu{=}0.8, \sigma{=}0.1)$	Low uncertainty
mushroom_medium	8,416	120	$N(\mu{=}0.5, \sigma{=}0.2)$	Moderate uncertainty
mushroom_low	8,416	120	$N(\mu{=}0.3, \sigma{=}0.15)$	High uncertainty

The mushroom dataset is from the UCI Machine Learning Repository. Uncertain variants are generated using the `DatasetConverter` utility with clamped normal distributions, following methodology from [5] and [10].

5.3 Configurations

Static minsup mode: Vary minsup across appropriate ranges for each dataset.

Top-K mode: $K \in \{5, 10, 20, 50\}$.

For each configuration we record: time (ms), memory (MB), nodes explored, pruning counts, and number of MFIs.

5.4 Correctness Validation

Cross-algorithm agreement (UGenMax and UFPMax produce identical MFI sets) and hand verification on contextUncertain.

5.5 Performance Measurement

PerformanceTracker uses `System.nanoTime()` for timing and `Runtime.totalMemory() - Runtime.freeMemory()` for memory, with GC forced before each run.

Bibliography

[1] Agrawal, R., Imielinski, T., and Swami, A. (1993). Mining association rules between sets of items in large databases. *Proc. ACM SIGMOD*, pp. 207–216.

[2] Agrawal, R. and Srikant, R. (1994). Fast algorithms for mining association rules. *Proc. VLDB*, pp. 487–499.

[3] Aggarwal, C.C., Li, Y., Wang, J., and Wang, J. (2009). Frequent pattern mining with uncertain data. *Proc. ACM SIGKDD*, pp. 29–38.

[4] Bayardo, R.J. (1998). Efficiently mining long patterns from databases. *Proc. ACM SIGMOD*, pp. 85–93.

[5] Chui, C.K., Kao, B., and Hung, E. (2007). Mining frequent itemsets from uncertain data. *Proc. PAKDD*, pp. 47–58.

[6] Fournier-Viger, P., et al. (2014). SPMF: A Java open-source pattern mining library. *JMLR*, 15, pp. 3569–3573.

[7] Gouda, K. and Zaki, M.J. (2005). GenMax: An efficient algorithm for mining maximal frequent itemsets. *Data Mining and Knowledge Discovery*, 11(3), pp. 223–242.

[8] Grahne, G. and Zhu, J. (2003). Efficiently using prefix-trees in mining frequent itemsets. *Proc. IEEE ICDM Workshop on FIMI*.

[9] Han, J., Pei, J., and Yin, Y. (2000). Mining frequent patterns without candidate generation. *Proc. ACM SIGMOD*, pp. 1–12.

[10] Leung, C.K.-S., Mateo, M.A.F., and Brajczuk, D.A. (2008). A tree-based approach for frequent pattern mining from uncertain data. *Proc. PAKDD*, pp. 653–661.

[11] Leung, C.K.-S. and Tanbeer, S.K. (2013). Fast tree-based mining of frequent itemsets from uncertain data. *Proc. DASFAA*, pp. 272–287.

[12] Li, H. and Zhang, N. (2019). Mining top-k frequent itemsets from uncertain databases. *Proc. DSAA*.

[13] Pan, F., Cong, G., Tung, A.K.H., Yang, J., and Zaki, M.J. (2003). CARPENTER: Finding closed patterns in long biological datasets. *Proc. ACM SIGKDD*, pp. 637–642.

[14] Szathmary, L., Napoli, A., and Valtchev, P. (2006). Towards rare itemset mining. *Proc. IEEE ICTAI*.